Cairo University

Faculty of Computers and Artificial Intelligence

The Prince's Rebellion

Console Adventure Game

01.03.2026

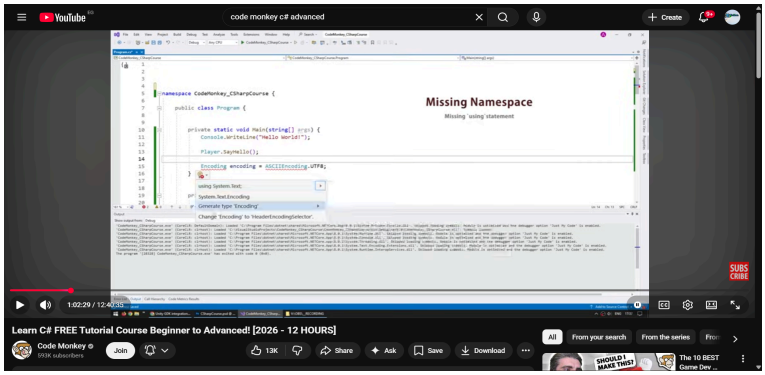
Omar Abdelaziz Ali Mohamed
ID: 20240375

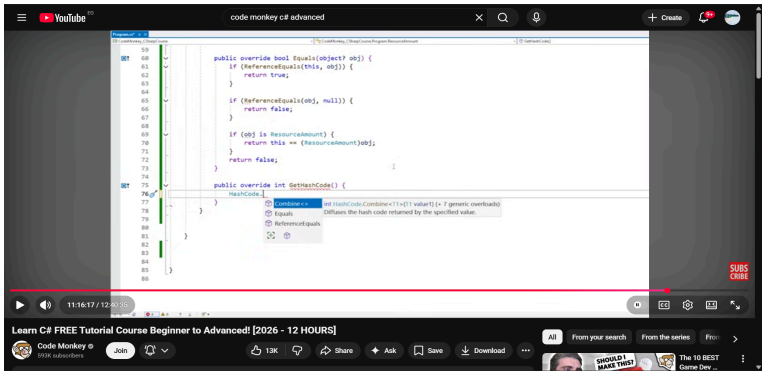
Learning Resource: [Learn C# FREE Tutorial Course Beginner to Advanced! [2026 - 12 HOURS]]

Demo Video: Project_Explanation.mp4

→ **Drive link** ←

→ **GitHub Repo** ←





1. Introduction

This project represents my first complete console-based game developed using standard C#. Although I am already involved in the game development field, this is the first time I built a full narrative-driven experience using pure C# outside of a game engine like Unity.

The purpose of this project was not only to create a playable story-driven adventure, but also to strengthen my understanding of core C# principles, Object-Oriented Programming (OOP), and software structure without relying on engine abstractions.

The game was also a refactored version of a previous implementation I originally built in Python. Rebuilding it in C# allowed me to compare paradigms, structure code more efficiently, and apply stronger architectural patterns.

2. Learning Background

Before building this project, I completed the 12-hour C# course by CodeMonkey, which covers C# fundamentals and practical implementation for game development.

From this course, I gained solid understanding of:

- Variables and Data Types
- Control Flow (if statements, loops)
- Methods and Parameters
- Classes and Objects
- Object-Oriented Programming Principles
- Encapsulation
- Inheritance
- Abstraction
- Delegates and Actions
- Basic Design Patterns
- Clean Code Structure
- Game logic structuring in C#

This project served as practical reinforcement of everything I studied, but in a pure console environment rather than Unity.

3. Project Overview

The game is a story-driven adventure where the player takes the role of a prince who rebels against his unjust father, the king.

The story flows through multiple phases:

1. Narrative introduction (cutscene style)
2. Betrayal and imprisonment
3. Ship escape gameplay
4. Random room exploration
5. Combat system
6. Final boss battle against the King

The gameplay combines:

- Decision-making
- Randomized events
- Turn-based combat
- Resource management (Health, Shield, Ammo, Coins)
- Progress-based objective system (Freeing prisoners before boss fight)

4. Core Programming Concepts Applied

4.1 Object-Oriented Programming (OOP)

This project heavily relies on OOP principles.

Abstraction

I created an abstract base class:

```
public abstract class Character
    {
        public string? Name { get; set; }
        public int Health { get; set; }
        public bool IsAlive => Health > 0;

        public abstract void ApplyDamage(int amount);
    }
```

This class defines shared properties:

- Name
- Health
- IsAlive
- ApplyDamage() (abstract method)

This ensures all characters must implement their own damage logic.

Inheritance

Two main classes inherit from Character:

- Player
- Enemy

This allowed shared structure while customizing behavior.

Encapsulation

Player properties like Shield, Coins, Ammo are encapsulated inside the Player class.

Damage handling logic is controlled through ApplyDamage(), preventing direct uncontrolled modification.

Polymorphism

ApplyDamage() is overridden differently in:

- Player (shield absorbs damage first)

```
public class Player : Character
    {
        public int Shield { get; set; } = 0;
        public int Coins { get; set; } = 0;
        public int Ammo { get; set; } = 0;
        public int PrisonersRemaining { get; set; } = 25;

        public Player(string name)
        {
            Name = name;
            Health = 50;
        }

        // Custom damage logic to account for the shield
        public override void ApplyDamage(int amount)
        {
            if (Shield >= amount)
            {
                Shield -= amount;
            }
            else
            {
                amount -= Shield;
                Shield = 0;
                Health -= amount;
            }
        }
    }
```

- Enemy (direct health reduction)

```
public class Enemy : Character
    {
        // Damage boundaries for scalable combat
        // ده Singleton جوى هو اللى علمنى ال
        public int GunDmgMin { get; set; }
        public int GunDmgMax { get; set; }
        public int SwordDmgMin { get; set; }
        public int SwordDmgMax { get; set; }
        public int DefendDmgMin { get; set; }
        public int DefendDmgMax { get; set; }

        public int PlayerGunMin { get; set; }
        public int PlayerGunMax { get; set; }
        public int PlayerSwordMin { get; set; }
        public int PlayerSwordMax { get; set; }

        public override void ApplyDamage(int amount)
        {
            Health -= amount;
        }
    }

```

This demonstrates runtime polymorphism.

4.2 Delegates and Action

I used:

```
public Action<Player, GameEngine>? ExecuteEvent { get; set; }
```

inside the RoomEvent class.

This allowed dynamic event execution from a list of possible room events.

It simulates an event system similar to how Unity handles actions and callbacks.

4.3 Data Structures

I implemented:

- List as an event pool
- Random for procedural variation

This enables dynamic room generation and unpredictable gameplay.

4.4 Game Loop Logic

The ship gameplay uses a loop:

```
while (player.IsAlive)
            {
                Console.Clear();
                roomNumber++;
                Console.WriteLine($"HP => {player.Health} | Shield => {player.Shield} | Ammo => {player.Ammo} | Coins => {player.Coins} | Tied prisoners => {player.PrisonersRemaining}");
                Console.WriteLine(new string('=', 68));
                DisplayText($"You found room-{roomNumber}!!");

                Console.Write("1- Go to the roof of the ship  2- Enter The Room  3- Skip the room ==> ");
                string choice = Console.ReadLine() ?? "";

                if (choice == "1")
                {
                    if (player.PrisonersRemaining <= 0)
                    {
                        // البوس الكبير (سينيور مجهود وانا بهرب منه ومن كورس الجرافيكس)
                        DisplayText("You have to fight the captain of the ship!");
                        bool win = StartBattle(player, CreateCaptain());
                        if (win)
                        {
                            DisplayText("You win the fight!!");
                            return 0; // Success code
                        }
                        return -1; // Loss code
                    }
                    else
                    {
                        DisplayText("Find all Prisoners first!!");
                    }
                }
                else if (choice == "2")
                {
                    if (eventPool != null)
                    {
                        RoomEvent randomEvent = eventPool[rng.Next(eventPool.Count)];

                        // Logic check for empty rooms
                        if (randomEvent.Description.Contains("prisoners") && player.PrisonersRemaining <= 0)
                        {
                            DisplayText("Empty room!!");
                        }
                        else
                        {
                            DisplayText(randomEvent.Description);
                            randomEvent.ExecuteEvent?.Invoke(player, this);
                        }
                    }
                }
                else if (choice == "3")
                {
                    continue;
                }
                else
                {
                    DisplayText("Invalid input. Please try again.");
                }
            }
```

This creates a continuous gameplay cycle until the player dies or completes objectives.

It simulates a simplified version of a game loop.

5. Combat System Design

The combat system is turn-based and includes:

- Gun attack (consumes ammo)
- Sword attack
- Defend

Damage ranges are dynamically calculated using Random within defined min/max ranges.

Each enemy type has scalable stats:

- Gun damage range
- Sword damage range
- Defensive damage range
- Player damage range

This allows easy balancing and future scalability.

6. Refactoring from Python to C#

Originally, this game concept was built in Python.

During refactoring, I learned:

- C# enforces stronger type safety
- OOP structure is more explicit
- Delegates provide more structured event handling
- Code organization is cleaner with namespaces and classes
- Method overriding is more formalized

Compared to Python, C# required more structured planning but resulted in more maintainable architecture.

7. Game Engine Structure

The GameEngine class works as the central controller.

Responsibilities include:

- Story narration
- Event initialization
- Ship gameplay logic
- Combat management
- Enemy factory creation

This follows a simplified version of the Single Responsibility Principle, even though everything is console-based.

8. Application Entry Point (Main Method)

In C#
Every console application starts execution from the `Main` method. It serves as the entry point of the entire program.

In this project, the `Main` function is intentionally kept minimal and clean in order to follow proper architectural design principles.

```
static void Main(string[] args)
        {
            GameEngine engine = new GameEngine();
            engine.Start();
        }
```

The method performs two main responsibilities:

1. It creates an instance of the `GameEngine` class, which initializes all core systems of the game such as event handling, gameplay flow, and combat mechanics.

2. It calls the `Start()` method, which begins the narrative introduction and transitions into the interactive gameplay.

The `Main` method does not contain any gameplay logic directly. Instead, it delegates responsibility to the `GameEngine`. This design choice improves separation of concerns and keeps the application entry point simple and maintainable.

9. Challenges Faced

1. Designing a balanced combat system
2. Handling invalid user input
3. Managing game pacing using Thread.Sleep
4. Structuring story and gameplay together
5. Avoiding repetitive logic
6. Making random events meaningful
7. Transitioning mindset from Unity-based thinking to pure C#

10. Skills Strengthened

Through this project, I strengthened:

- OOP design
- Clean code organization
- Logical thinking
- Game system architecture
- Combat balancing
- Refactoring skills
- Understanding of standard C# outside Unity
- Console application flow management

11. Why This Project Matters

Even though I already work in the game development field, this project was important because:

- It reinforced core C# knowledge
- It forced me to build systems from scratch
- It improved my understanding of architecture
- It increased confidence in building standalone applications
- It helped me deeply understand how engines abstract complexity

Now, when working in Unity, I better understand what happens behind the scenes.

12. Future Improvements

If I continue developing this project, I would:

- Implement inventory system
- Add leveling system
- Add save/load system
- Improve combat AI
- Separate files into multiple source files
- Apply Dependency Injection
- Add logging system
- Convert it into a graphical version

13. Conclusion

This console adventure game represents a significant milestone in my C# learning journey.

It is my first fully structured console game written using standard C#, built after completing the 12-hour C# course by CodeMonkey.

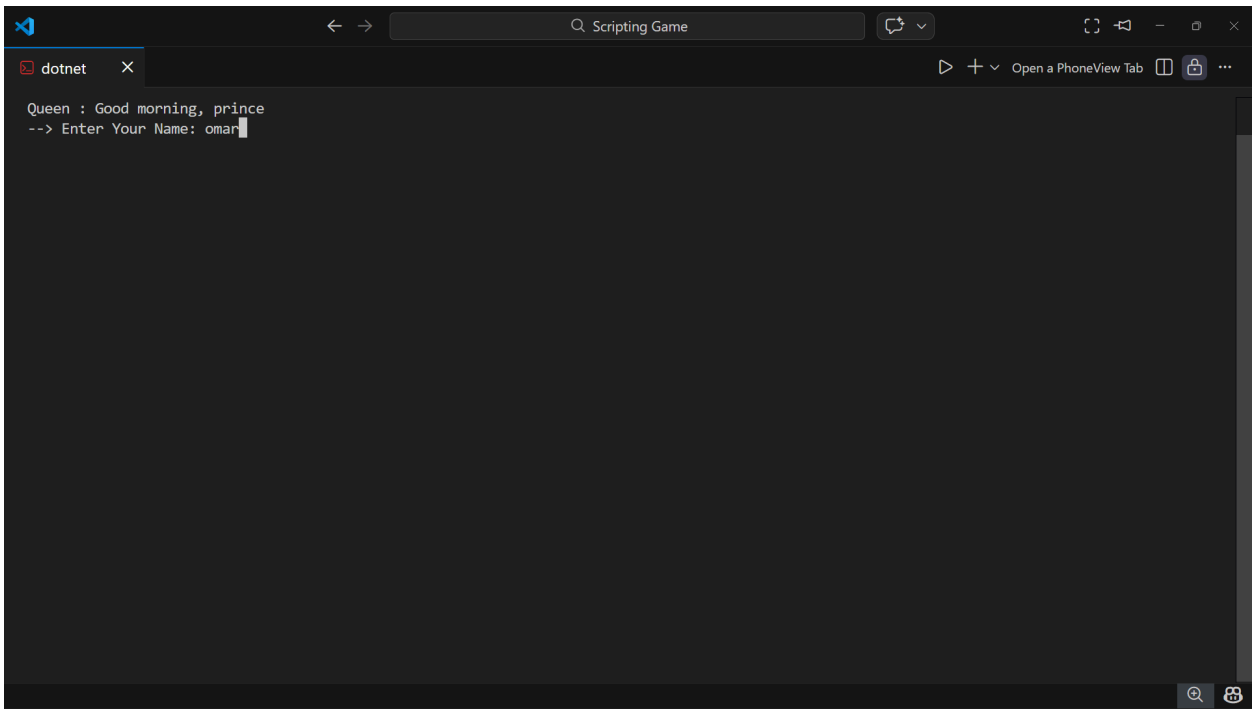
The project demonstrates:

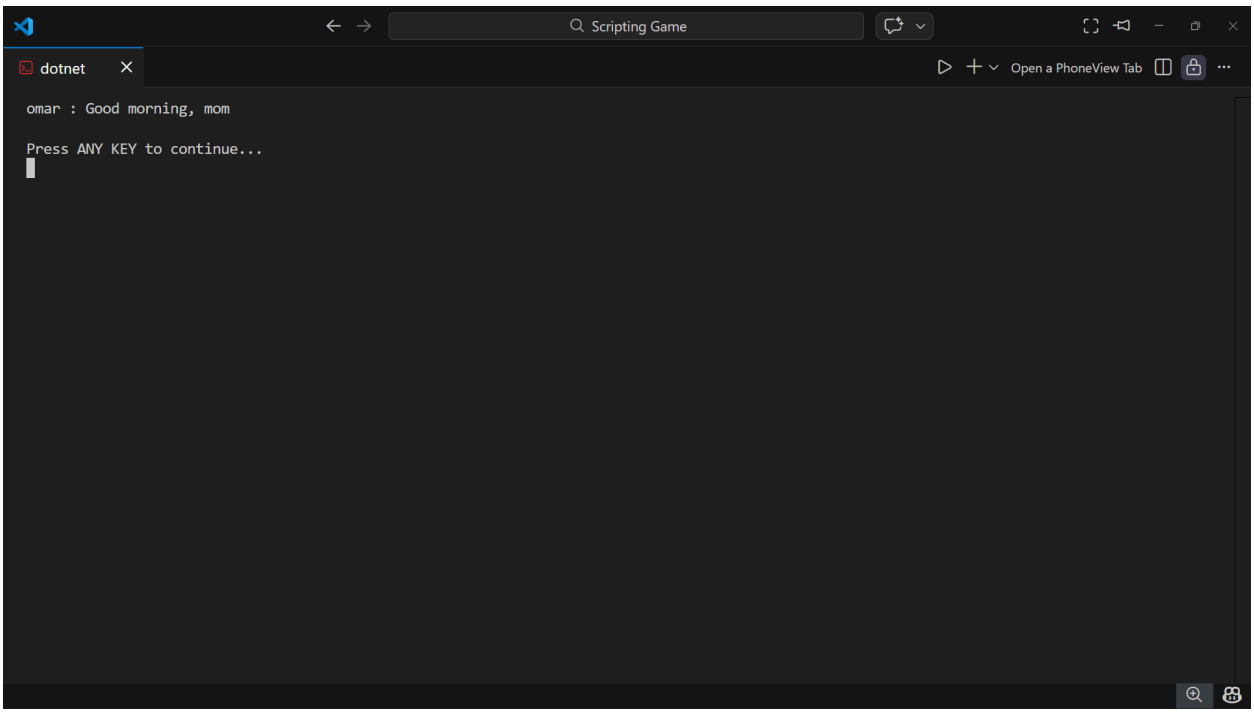
- Strong understanding of OOP
- Clean architecture principles
- Event-driven logic
- Combat system design
- Refactoring from Python to C#

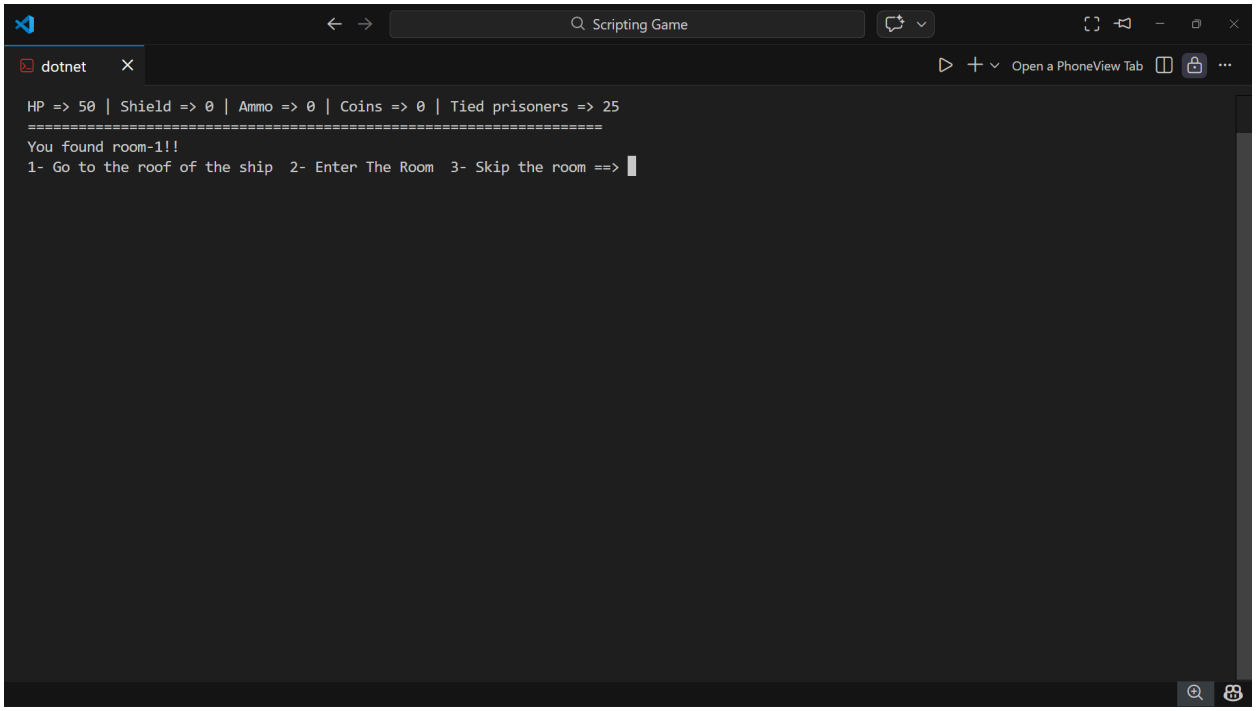
Most importantly, it shows my ability to transform knowledge into a complete, playable system.

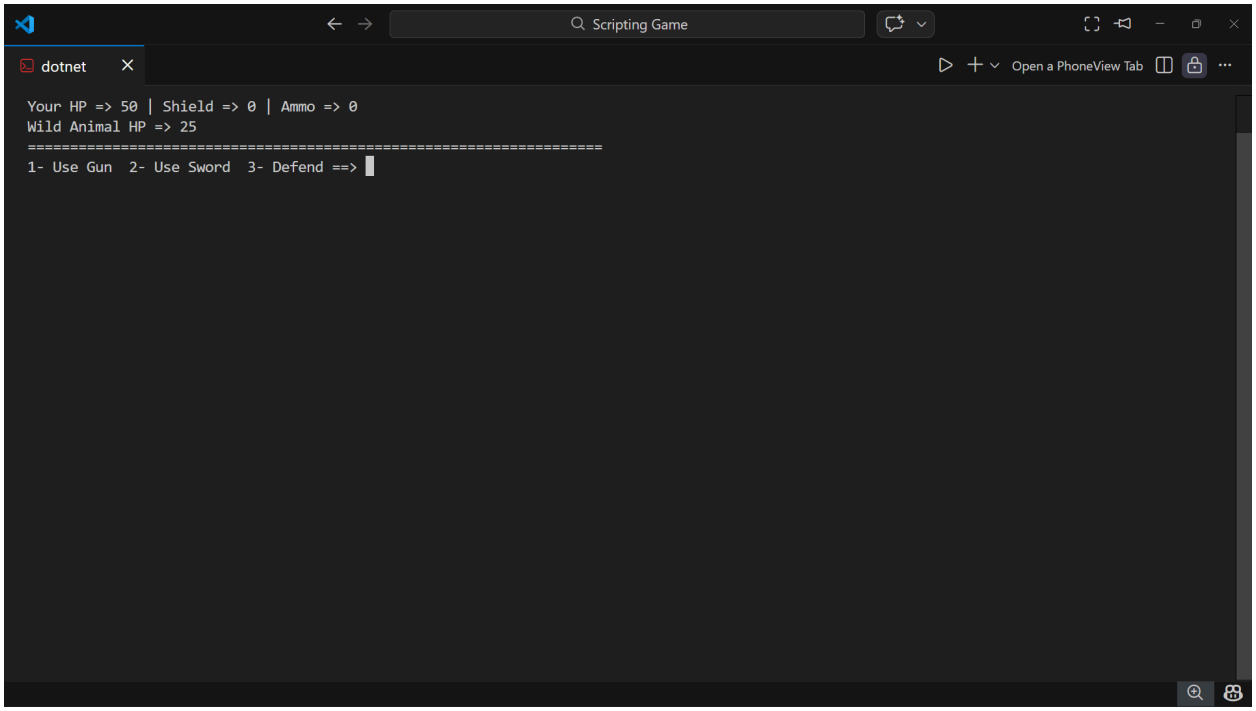
This project is not just a console game — it is proof of structured learning, disciplined implementation, and growth as a game developer.

PROJECT GALLERY






Scripting Game
dotnet
Open a PhoneView Tab
HP => 50 | Shield => 0 | Ammo => 0 | Coins => 0 | Tied prisoners => 25
==========
You found room-1!!
1- Go to the roof of the ship 2- Enter The Room 3- Skip the room ==>


Scripting Game
dotnet
Open a PhoneView Tab
Your HP => 50 | Shield => 0 | Ammo => 0
Wild Animal HP => 25
==========
1- Use Gun 2- Use Sword 3- Defend ==>

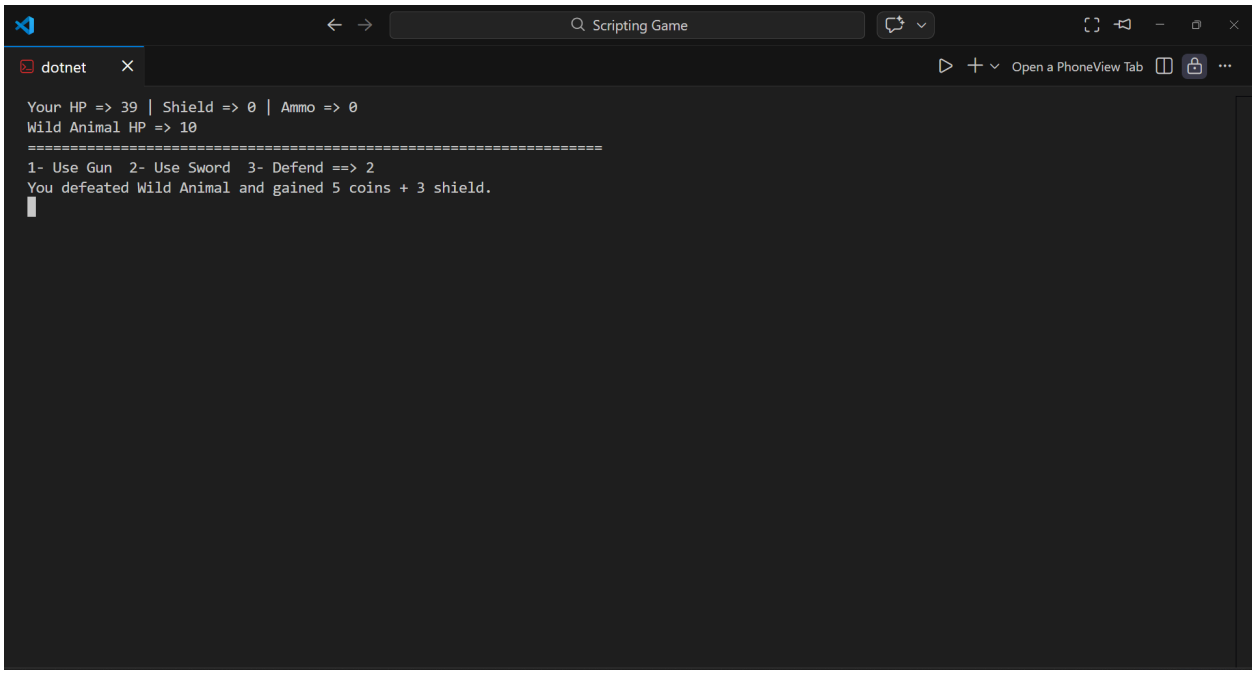

Scripting Game
dotnet
Open a PhoneView Tab
Your HP => 39 | Shield => 0 | Ammo => 0
Wild Animal HP => 10
==========
1- Use Gun 2- Use Sword 3- Defend ==> 2
You defeated Wild Animal and gained 5 coins + 3 shield.

Table Of Content

Section	Page Number
The Prince's Rebellion	
1. Introduction	1
2. Learning Background	2
3. Project Overview	3
4. Core Programming Concepts	4 - 8
5. Combat System Design	9
6. Refactoring from Python	9
7. Game Engine Structure	10
8. Application Entry Point (Main Method)	10
9. Challenges Faced	11
10. Skills Strengthened	12
11. Why This Project Matters	12
12. Future Improvements	13
13. Conclusion	13
PROJECT GALLERY	14